COMP232 Lab 6

HMAC Message Authentication

This lab exercise is about the hash-based authentication code `HMAC-SHA256` that uses a secret key. The key itself is assumed to be a shared secret that has already been established between the sender and receiver.

References:

- JCA/JCE Reference Manual
- Python HMAC Reference Manual
- Python hashlib Reference Manual
- HMAC - Wikipedia

1. Understand `HMAC-SHA256`

Compile and run the program `initMac.java` (for Python download and run `HMAC.py`).

Study the code and see how Message Authentical Code (MAC) generation is implemented. It is in fact an implementation of the authentication scheme presented in Lecture 9.

More details on this authentication scheme can be found at https://en.wikipedia.org/wiki/HMAC

Modify the code to show that indeed, it computes MAC:

- If a Verifier uses the same secret key to initialize his/her MAC object and recalculates MAC code of the same text then the result will be the same.
- If a Verifier uses the same secret key, but calculates MAC code of a different text then the result will be different.
- If a Verifier uses a different secret key, but calculates MAC code of the same text then... (complete the sentence)

2. Compare, and contrast

Compare HMAC-SHA256 with hash functions (e.g. SHA-256), and the authentication schemes considered in LAB 5, in terms which of these methods can be used to establish:

- Message integrity.
- Authentication (the origin of the message can be proved to the receiver).
- Non-repudiation (the origin of the message can be proved to a third party, not only to the receiver).

3. Combine with Diffie-Hellman (after the lab, *optional!*)

Since HMAC assumes that the sender and recipient share a secret key already, you should combine the Diffie-Hellman protocol (that you learned in Lab 4) with HMAC-SHA256.
More precisely:

- Write a program that first executes Diffie-Hellman to establish a shared secret key `K`. Then use `K` when creating an HMAC-SHA256 authentication code. (You might want to first learn about the relationship between key length and block size from the Wikipedia link above.)
- As a trivial extension, take user input for the message `M`. Then compute the HMAC hash for `M` with key `K`. Put all of them together for the final packet that contains the message and the hash.